Neurocomputing 607 (2024) 128356

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

GraphixMatch: Improving semi-supervised learning for graph classification with FixMatch

Eunji Koh, Young Jae Lee, Seoung Bum Kim *

School of Industrial and Management Engineering, Korea University, 145, Anam-ro, Seongbuk-gu, Seoul 02841, Republic of Korea

ARTICLE INFO

Communicated by Z. Tao

Keywords:
FixMatch
Graph augmentation
Graph classification
Graph neural network
Semi-supervised learning

ABSTRACT

Graph classification is a critical problem that predicts the properties of an entire graph representing complicated relationships. This problem has been extensively studied in a supervised manner in various industries via graph convolutional networks that required large numbers of labeled graphs. However, labeled graphs are typically expensive and time-consuming to obtain in the real-world. Training graph classification models is difficult because of the sparseness of labeled data. Therefore, semi-supervised learning (SSL) algorithms that make full use of unlabeled data and overcome the lack of labels have led to the expansion of graph classification studies. In this study, we propose GraphixMatch to learn the graph properties of both labeled and unlabeled graphs based on the use of FixMatch for graph classification. To effectively capture the characteristics of a graph, we newly define and apply weak and strong augmentation of the graph. We evaluate the proposed method on seven benchmarks from a public dataset. We show that compared with various SSL methods, the simplified proposed method with weak and strong augmentation is superior. Furthermore, we conduct extensive additional experiments to derive the important parameters of GraphixMatch.

1. Introduction

Because graphs are widely used in various fields such as chemistry, biology, and material science, graph classification has emerged as an important issue [1–3]. Graph classification aims to predict the labels of an entire graph by learning its inherent properties that represent complicated relationships [4–7]. For example, graph classification has been adopted in chemistry to predict the properties of new compounds and the properties of drugs [8,9]. Traditionally, graph classification approaches have relied on hand-crafted features extracted from graphs based on experts' judgment [10]. However, such features can only capture the superficial nature of graphs. Therefore, more effective approaches are required to definitively capture the inherent characteristics of the attributes of nodes and the structural information of the edges contained in a graph [11].

Recently, deep learning methods have been studied in a supervised learning manner for their powerful representational capability based on raw data [12–15]. Inspired by convolutional neural networks (CNN) [16] that can learn inherent features of image data, graph convolutional networks (GCN) [17] have been developed by using the neighbor-aware message passing mechanism in the neighbor aggregation process. This mechanism allows GCN to capture the inherent features of nodes and edges. Since the advent of GCN, graph isomorphism networks (GIN) [18] have been studied to solve the problem of excessive smoothing of the representations of the graphs. Experiments have demonstrated that GINs are superior to other graph neural networks in their discriminative representational capability [18].

In general, supervised learning methods require large amounts of labeled data to achieve good performance. However, collecting large-scale and accurately labeled graph data is difficult [9]. Manually annotating graphs is typically expensive and time-consuming. In particular, the cost of annotations can be extremely high when the labeling has to be done by domain experts. For example, generating a single label in the chemical domain requires expensive density function theory (DFT) computations of over five hours duration [19]. Furthermore, the data may be labeled inaccurately because the decisions of experts can be biased based on their levels of experience or knowledge. These issues complicate the collection of large-scale labeled graph data and hinder the application of supervised learning approaches in graph classification.

Semi-supervised learning (SSL) has emerged as a powerful approach to overcome the lack of labeled data and the limitations of supervised learning for training models [20–25]. SSL can be applied to labeled and unlabeled data via a variety of methods such as pseudo-labels [26] and consistency regularization [27]. For example, the pseudo-label method generates artificial labels and trains a model to predict the artificial

* Corresponding author.
E-mail address: sbkim1@korea.ac.kr (S.B. Kim).

https://doi.org/10.1016/j.neucom.2024.128356
Received 4 June 2023; Received in revised form 27 April 2024; Accepted 10 August 2024
Available online 13 August 2024

labels when given unlabeled data as input [24]. At the same time, a small amount of labeled data is used to train the model to predict the true labels. Many methods based on SSL have shown superior performance in various tasks. In particular, SSL has performed well in the field of computer vision by leveraging large amounts of unlabeled data [20,21,24]. Recently, inspired by the success of SSL in various fields, SSL approaches for graph classification have been studied to achieve better performance and overcome the lack of labeled graphs [9,28,29].

In this study, we propose GraphixMatch, which combines graph augmentation and FixMatch [24] in a way known as a simple hybrid SSL method for image data. FixMatch is a significant simplified algorithm that follows the both approach of pseudo-labeling and consistency regularization to use unlabeled data. Pseudo-labeling and consistency regularization are the important components to use the unlabeled data by relying on data augmentations and the model itself to obtain artificial labels [24]. We redefine graph-specific weak augmentation and strong augmentation as a prerequisite to apply FixMatch to graph data. We use node-drop for weak augmentation and subgraph for strong augmentation. Node-drop [30] randomly masks some nodes, producing a weakly distorted version of the given graphs. Subgraph [30] masks both nodes and edges to produce a strongly distorted version of the given graphs. GraphixMatch, unlike various SSL comparison methods, is a simple SSL algorithm without complex loss conditions and numerous hyperparameters. We evaluate the proposed method on the following seven publicly accessible graph datasets: PROTEINS [31], IMDB-B, IMDB-M, REDDIT-B, REDDIT-M-5K, COLLAB [32], DD [33]. We show that GraphixMatch not only outperforms a fully supervised baseline trained on only labeled data, but also outperforms other SSL methods in graph classification when the availability of labeled data is limited.

The main contributions of our method can be summarized as follows:

- We propose a simple SSL algorithm without complex loss components and numerous hyperparameters, unlike various graph-specific SSL methods. It improved efficiency by employing FixMatch which is powerful in image data.
- We define graph-specific weak augmentation and strong augmentation. We provide extensive studies of augmentation strategies to demonstrate the effect of augmentation on graphs. To the best of our knowledge, this has not been previously studied.

The remainder of the paper is organized as follows. Section 2 consists of a review of SSL studies, including graph-specific methods. Section 3 illustrates the details of the proposed method. Experimental results and discussion are presented in Section 4. Finally, Section 5 provides the conclusion and directions for future work.

2. Related works

2.1. Semi-supervised learning

SSL aims to learn powerful representations by leveraging unlabeled data. The main mechanisms of SSL are consistency regularization and entropy minimization [27]. Consistency regularization leverages unlabeled data based on an assumption that the model should calculate similar predictions when given different perturbed versions of the same data [24]. Therefore, this allows the models to consistently output predictions when fed semantically similar data as input. For example, π-model [25], mean-teacher [25], and virtual adversarial training (VAT) [23] have been proposed as a way to use consistency regularization to learn a representation that prevents overfitting of a small amount of labeled data. Entropy minimization [22] assumes the decision boundary obtained from the training models should be located in a low entropy space. Therefore, entropy minimization has been adopted to bolster the confident predictions of models on unlabeled data.

More recently, a novel stream of SSL has built hybrid approaches [20,21,24]. The core idea behind the hybrid approaches is to combine different mechanisms such as consistency regularization and entropy minimization. For example, MixMatch [21] performed well by using both entropy minimization and data augmentation. To overcome the complexity of MixMatch, FixMatch [24] uses both consistency regularization and pseudo-labeling to further improve performance. FixMatch can transform the complicated loss function of MixMatch into a simple loss function by using pseudo-labeling. Surprisingly, hybrid SSL approaches that use only small amounts of labeled data have demonstrated their effectiveness by outperforming the supervised learning approaches that leverage many labeled data [20,21,24].

2.2. Graph-specific semi-supervised learning

Driven by the growing interest in graph classification, many graph-specific SSL approaches have been proposed. For example, Sun et al. [29] proposed InfoGraph that learns representations via maximizing the mutual information between the representations of a whole graph and its substructures. Hao et al. [9] proposed an active semi-supervised graph neural network (ASGNN) by adopting a teacher–student framework that transfers knowledge from a teacher model to a student model. You et al. [30] explored the impact of different graph augmentation combinations on multiple datasets and designed a graph contrastive learning (GraphCL) framework. To improve GraphCL, You et al. [34] proposed joint augmentation optimization (JOAO), which automatically selects data augmentation. Ju et al. [28] proposed a graph harmonic neural network (GHNN) that effectively uses unlabeled graphs by using novel harmonic contrastive loss and harmonic consistency loss. In addition, GHNN used both GCN [17] and graph kernel network (GKN) [32] modules to fully learn the inherent features of graphs. The result showed that the representation capability of GHNN was superior to others. The graph-specific SSL approaches are often superior to traditional SSL approaches to graph classification [28]. However, these methods are limited in that they require numerous hyperparameters and are more complicated because of the complex loss terms and combination modules involved in capturing the inherent features of a graph.

3. Methodology

We propose GraphixMatch, which is a graph-specific SSL approach based on FixMatch. The core idea of GraphixMatch is the use of a combination of consistency regularization and pseudo-labeling as well as separate weak and strong augmentations [24]. Pseudo-labeling and consistency regularization are significant to obtain artificial labels for the unlabeled data by relying on the model itself and the perturbed data [24]. Specifically, GraphixMatch redefines weak and strong augmentation for graph data. In this section, we describe the proposed method in detail and review the graph augmentation used by GraphixMatch.

We denote $G = \{(g_b, l_b) : b \in (1, \ldots, B)\}$ as a batch B of labeled graphs, where g_b is the training graph and l_b is the label. On the other hand, $U = \{(u_b) : b \in (1, \ldots, \mu B)\}$ means a batch of μB unlabeled graphs. u_b is a hyperparameter to determine the relative sizes of G and U. $p_m(y|g)$ represents the predicted class distribution calculated by the model when g is the input value and the model denotes GCN to capture the inherent features of nodes and edges. Specifically, to calculate $p_m(y|g)$, the model aggregates every node representation of input graph and then extracts graph representation. First, the model aggregates node representations that capture structural information by updating node representations over i iterations with multi-layer perceptron (MLP). Let N denote the set of nodes of the entire graph

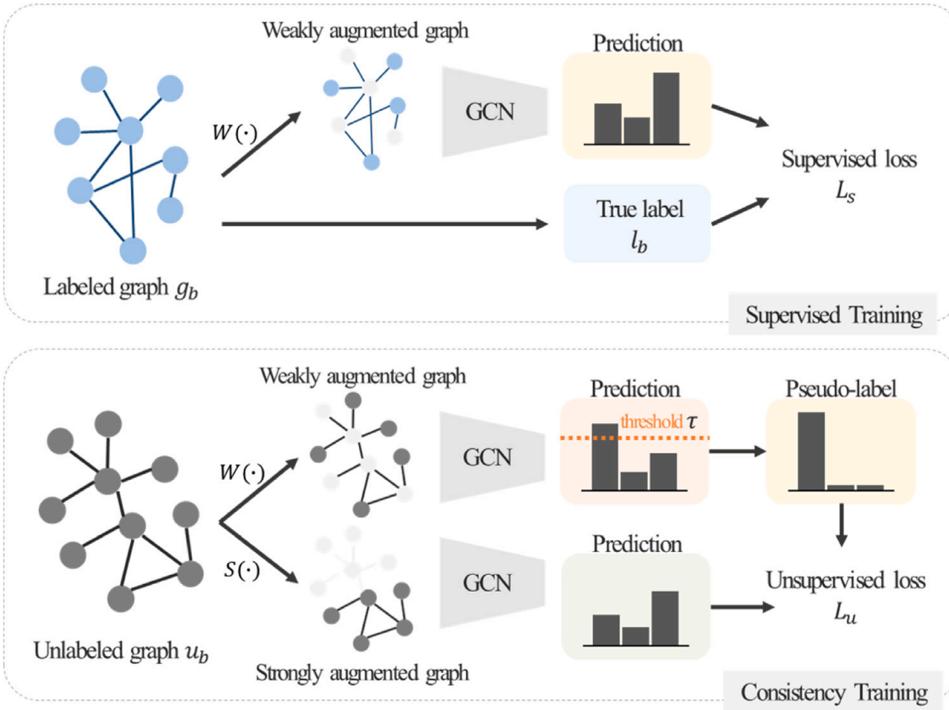

Fig. 1. Overall framework of GraphixMatch consisting of supervised and consistency training.

and denote each node $n \in N$. The node representation of ith iteration $h_n^{(i)}$ is as follows:

$$h_n^{(i)} = MLP^{(i)}((1+\epsilon^{(i)}) \cdot h_n^{(i-1)} + \Sigma_{z \in V(n)} h_z^{(i-1)}), \tag{1}$$

where $V(n)$ is the set of adjacent nodes to n, and ϵ is a learnable parameter. After i iterations of aggregation, the model extracts a representation of the entire graph with a readout function that uses the node representation to generate an embedding of the entire graph. Formally, the representation of the entire graph h_g is as follows:

$$h_g = CONCAT(READOUT(\{h_n^{(i)} | n \in N\} | i = 0, 1, \dots, I)). \tag{2}$$

Finally, the model computes $p_m(y|g)$ using MLP to predict the label of the input graph.

$$p_m(y|g) = MLP(h_g). \tag{3}$$

$C(r,t)$ represents the cross-entropy between two probability distributions r and t. In addition, we denote weak and strong augmentations as $W(\cdot)$ and $S(\cdot)$, respectively.

Fig. 1 shows the overall framework of GraphixMatch. In Fig. 1, GraphixMatch consists of two parts: supervised training and consistency training. Part of the supervised training aims to predict true labels with labeled graphs. To overcome overfitting for the scarcity of labeled graphs, GraphixMatch uses weak augmentation to produce weakly distorted graphs $W(g_b)$. Then, the weakly augmented graph $W(g_b)$ is provided as input to the model. In the supervised training process, we define a supervised loss L_s by applying the cross-entropy loss to weakly augmented labeled graphs. Supervised loss is as follows:

$$L_s = \frac{1}{B} \Sigma_{b=1}^{B} C(l_b, p_m(y|W(g_b))). \tag{4}$$

Consistency training aims to use unlabeled graphs together with consistency regularization and pseudo-labeling. Consistency regularization is applied based on an assumption that the model's predictions should be similar when given various augmented versions of the same data [27]. To apply consistency regularization, GraphixMatch uses weak and strong augmentations to produce weakly distorted graph $W(u_b)$ and strongly distorted graph $S(u_b)$, respectively. Then, we give the generated distorted graphs as input to calculate the predictions $p_m(y|W(u_b))$ and $p_m(y|S(u_b))$.

In general, consistency regularization uses an additional loss term to avoid deviating from the true label when training the model to predict consistently based on perturbed versions of the same data. However, to overcome the complexity of the loss function, GraphixMatch applies pseudo-labeling to simply encourage model reliability. The idea behind pseudo-labeling is to use the model itself to obtain artificial labels for unlabeled data [26,27]. Specifically, it only uses artificial labels with the largest class probability greater than or equal to the predefined threshold τ. By using the threshold, the model can only use confident predictions for training and it prevents deviation of the predictions from the true label. Based on pseudo-labeling, GraphixMatch calculates artificial label $argmax(p_m(y|W(u_b)))$ using the weakly augmented graphs and predictions computed from a predefined threshold.

Finally, we define an unsupervised loss L_u by applying cross-entropy between the artificial labels from the weakly augmented graphs and the predictions of the model from the strongly augmented graphs. Unsupervised loss with artificial labels is as follows:

$$L_u = \frac{1}{\mu B} \Sigma_{b=1}^{\mu B} (f(o_b, \tau)) C(\hat{o_b}, p_m(y|S(u_b))). \tag{5}$$

where $f(o_b, \tau) = \begin{cases} 1 & \text{when } \max(o_b) \geq \tau \\ 0 & \text{O/W} \end{cases}$ and $o_b = p_m(y|W(u_b))$. Finally, to train GraphixMatch, the final loss function is simply defined as follows:

$$L_t = L_s + \lambda_u L_u, \tag{6}$$

where λ_u is a fixed hyperparameter for adjusting the relative weight of the unsupervised loss.

Augmentation is an important component that allows GraphixMatch to be applied effectively to graph data. This is because it differs crucially from the FixMatch, which is suitable for image data. The weak and strong augmentations previously defined in FixMatch are suitable for image data. For example, flip and shift, which is a weak augmentation for images leveraged in FixMatch, alters or randomly shifts the left and right sides of an image [35]. However, it is difficult

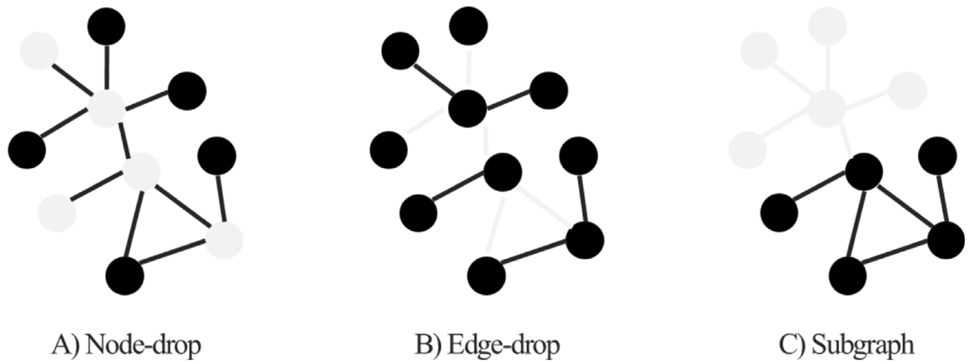

Fig. 2. Three types of augmentation for graph data. Light gray represents the masked portion of entire graph. All examples of augmentation use a masking ratio of 0.5.

to apply flip and shift to a graph because it is an isomorphic graph even if it is changed or moved left and right. Therefore, we redefine weak and strong augmentations suitable for graph data.

Fig. 2 shows the popular augmentations for graph data. In Fig. 2, the three types of augmentation are node-drop, edge-drop, and subgraph. Node-drop randomly masks specific parts of the entire node [30]. The core assumption of node-drop is that the semantic meaning of the entire graph is robust to missing parts of nodes [30]. Edge-drop perturbs the graph connection pattern by masking a certain ratio of edges. Edge-drop relies on an assumption that the semantic meaning of the entire graph is robust to changes in connection patterns [30]. Subgraph samples a specific part of the entire graph by using a random walk algorithm. The subgraph assumes that local structure can be preserved similar to the semantic meaning of the entire graph [30]. All augmentation strategies use predefined masking ratios. Among the various graph augmentations, we adopt node-drop and subgraph for weak and strong augmentations, respectively. Algorithm 1 summarizes the proposed GraphixMatch algorithm.

Algorithm 1 GraphixMatch

Denote labeled graph and unlabeled graph batches $G = \{(g_b, l_b) : b \in (1, \ldots, B)\}$, $U = \{(u_b) : b \in (1, \ldots, \mu B)\}$
Denote model parameter θ
Denote confidence threshold τ
Denote cross-entropy C
Denote unlabeled graph ratio and unsupervised loss weight μ, λ_u
Denote weak augmentation and strong augmentation W, S
while Training **do**
  $L_s = \frac{1}{B}\Sigma_{b=1}^{B} C(l_b, p_m(y|W(g_b)))$ ▷ Supervised loss for labeled graphs
  $o_b = p_m(y|W(u_b);\theta)$ ▷ Compute prediction after applying weak augmentation to unlabeled graphs
  $L_u = \frac{1}{\mu B}\Sigma_{b=1}^{\mu B}(f(o_b,\tau))C(\hat{o_b}, p_m(y|S(u_b))$, where $f(o_b,\tau) = \begin{cases} 1 & \text{when } \max(o_b) \geq \tau \\ 0 & \text{O/W} \end{cases}$ ▷ Unsupervised loss with pseudo-labeling and threshold
  Total $L_t = L_s + \lambda_u L_u$
end while

4. Experiments

In this section, we report the results from experiments performed on seven public graph benchmark datasets to demonstrate the benefits of GraphixMatch. We conducted various experiments using unlabeled data to evaluate representation capability. We compared the graph classification performance of GraphixMatch with previous methods. We also provided extensive studies of the augmentation strategies used in GraphixMatch to show augmentation effects on graphs. In addition, we demonstrated the effectiveness of our approach by comparing its classification performance with the recent SSL baseline by varying the amounts of unlabeled training data and thresholds.

Table 1
Overview of the seven datasets.

Datasets	Sizes	Avg. nodes	Avg. edges	Classes
PROTEINS	1113	39.06	72.82	2
IMDB-B	1000	19.77	96.53	2
IMDB-M	1500	13.00	65.94	3
REDDIT-B	2000	429.63	497.75	2
REDDIT-M-5K	4999	508.52	594.87	5
COLLAB	5000	74.49	2457.78	3
DD	1178	284.32	715.66	2

4.1. Dataset

We evaluated the performance of the proposed GraphixMatch on the following seven publicly accessible graph datasets: PROTEINS, IMDB-B, IMDB-M, REDDIT-B, REDDIT-M-5K, COLLAB, DD [31–33]. These datasets include different types of graphs such as molecules, social networks, and scholar connections. Specifically, there are two bioinformatics datasets, four social network datasets, and one on scientific paper collaboration [28]. Table 1 shows an overview of the seven datasets, including the average number of nodes and the average number of edges in each graph. Datasets generally consist of two or three classes; REDDIT-M-5K consists of five classes.

We randomly split the datasets into training, validation, and test sets in a 7:1:2 ratio to follow the semi-supervised scenario. Specifically, we sampled 28% of the training sets with labeled data and the rest of the training sets with unlabeled data for which a ground-truth label was unavailable. To demonstrate the effectiveness of the proposed method under the same conditions as the other SSL baselines, we set as out default the use of 50% of the labeled data in the training set for experimentation. Validation and test sets were used for hyperparameter selection and model performance evaluation, respectively.

4.2. Implementation details

To train GraphixMatch, we followed all the training conditions used in the previous study GHNN [28]. We adopted three-layer GIN [18] with one-layer sum-pooling as our GCN backbone. We trained GraphixMatch for 300 epochs with a batch size of 64. We set the threshold used to generate artificial labels to 0.95. The percentage of labeled data was fixed at 50%. In addition, we applied appropriate masking ratios found for each dataset through extensive experiments. To optimize GraphixMatch, we used Adam optimizer [36] with a learning rate of 0.01. We repeated all experiments five times and reported the average accuracies with standard deviations. All experiments were implemented with a single NVIDIA RTX 2080 Ti with PyTorch Geometric [37].

Table 2
Results of GraphixMatch and comparative methods. The highest accuracy for each dataset is in bold.

Methods		PROTEINS	IMDB-B	IMDB-M	REDDIT-B	REDDIT-M-5K	COLLAB	DD
Supervised learning	GCN	63.3 ± 1.4	63.4 ± 2.1	39.2 ± 1.6	69.8 ± 1.1	38.6 ± 2.5	61.7 ± 1.5	62.5 ± 1.5
	GCN (100%)	70.2 ± 1.1	69.7 ± 1.8	46.4 ± 1.4	77.3 ± 1.7	44.2 ± 2.1	67.1 ± 1.8	83.6 ± 1.5
Traditional semi-supervised learning	EntMin	62.7 ± 2.7	67.1 ± 3.7	37.4 ± 1.2	66.9 ± 3.5	38.7 ± 2.8	63.8 ± 1.6	59.8 ± 1.3
	phi-model	63.2 ± 1.2	67.0 ± 3.4	39.0 ± 3.5	67.1 ± 2.9	39.0 ± 1.1	63.7 ± 1.0	61.8 ± 1.8
	Mean-teacher	64.3 ± 2.1	66.4 ± 2.7	38.8 ± 3.6	68.7 ± 1.3	39.2 ± 2.1	63.6 ± 1.4	60.6 ± 1.8
	VAT	64.1 ± 1.2	67.2 ± 2.9	39.6 ± 1.4	70.8 ± 4.1	38.9 ± 3.2	64.1 ± 1.1	59.9 ± 2.6
Graph-specific semi-supervised learning	InfoGraph	68.2±.07	71.8 ± 2.3	42.3 ± 1.8	75.2 ± 2.4	41.5 ± 1.7	65.7 ± 0.4	67.5 ± 1.4
	ASGNN	67.7 ± 1.2	70.6 ± 1.4	41.2 ± 1.4	73.1 ± 2.3	42.2 ± 0.8	65.3 ± 0.8	68.5 ± 0.6
	GraphCL	69.4 ± 0.8	71.2 ± 2.5	43.7 ± 1.3	75.8 ± 1.7	42.3 ± 0.9	66.4 ± 0.6	68.7 ± 1.2
	JOAO	68.7 ± 0.9	71.0 ± 1.9	42.6 ± 1.5	74.8 ± 1.6	42.1 ± 1.2	65.8 ± 0.4	67.9 ± 1.3
	GHNN	71.1 ± 0.3	72.3 ± 0.6	42.8 ± 0.4	76.3 ± 0.7	44.1 ± 0.5	67.1 ± 0.3	70.6 ± 0.4
	GraphixMatch	**75.0 ± 0.9**	**73.9 ± 1.2**	**45.3 ± 2.0**	**83.8 ± 2.5**	**46.8 ± 1.4**	**69.2 ± 0.3**	**76.0 ± 5.4**

4.3. Semi-supervised scenario

We evaluated the proposed method in a semi-supervised scenario with few labeled data. The semi-supervised scenario used only 50% of the labeled data. We compared GraphixMatch with the fully supervised model and the other SSL methods to demonstrate the effectiveness of GraphixMatch. Comparative methods included a fully supervised method, traditional SSL approaches, and graph-specific SSL approaches. Specifically, for the fully supervised method, we used three-layer GIN [18] with one-layer sum-pooling as the GCN model, which is the same as the GCN backbone of GraphixMatch. We trained GCN on a small amount of labeled data to evaluate the effectiveness of capturing the inherent features with limited labeled data. Furthermore, we also reported a GCN that used 100% of the labeled data. We compared performance with GraphixMatch using entropy minimization (EntMin), π-model, mean-teacher, and VAT for the traditional SSL approaches. Finally, we adopted InfoGraph, ASGNN, GraphCL, JOAO, and GHNN as comparison methods for the graph-specific SSL approaches. Table 2 shows the accuracy of GraphixMatch and comparison methods on seven datasets. Among the results presented in Table 2, the performance of all methods except GraphixMatch is the experimental result presented by GHNN [28].

As shown in Table 2, traditional SSL methods and graph-specific SSL methods showed equivalent or superior performance compared with the fully supervised method. In particular, graph-specific SSL methods outperformed traditional SSL methods because of complex loss terms or combinations of graph-specific architectures. However, GraphixMatch outperformed all comparative methods regardless of datasets despite the use of a simple loss term for training. For example, on the REDDIT-B dataset, GraphixMatch achieved 83.8% accuracy, which is significantly better than the 76.3% achieved by GHNN. Moreover, our method outperformed GCN using 100% of labeled data, despite the lack of labeled data. The success of GraphixMatch proved that graph-specific weak and strong augmentations contributed to the appropriate transformation to fit the graph data. Based on these results, we confirmed the effectiveness of GraphixMatch as an SSL approach to graph data.

4.4. Different proportions of labeled data

We conducted experiments on variations in the number of labeled data to evaluate the robustness of GraphixMatch. We experimented by reducing the proportion of labeled data from 100% to 75%, 50%, 25%, 5%, and 1%. In particular, when the number of labeled data is 1%, it is an extremely low-label scenario. Table 3 shows the performance for variations in the number of labeled data. GraphixMatch showed higher performance as the percentage of labeled data increases. However, we demonstrated that the proposed method is superior in that it uses only 50% of the labeled data in the semi-supervised scenario, similar to the comparison methods. GraphixMatch performed well even with extremely limited labeled data. Specifically, GraphixMatch outperformed the other SSL methods presented in Table 2, despite training with a small percentage of labeled data compared with other SSL methods. For example, on the REDDIT-B dataset containing 1% of labeled data for training, GraphixMatch had an accuracy of 76.6%, which outperformed comparative methods containing 50% labeled data as shown in Table 2.

We also compared GraphixMatch with GHNN for different proportions of labeled data in the training sets for the following datasets: PROTEINS, REDDIT-M-5K, and DD. Fig. 3 shows the performance of GraphixMatch and GHNN in orange and blue, respectively, when the proportion of labeled data in each dataset was changed. In Fig. 3, GraphixMatch outperformed GHNN for all proportions of the labeled data, except when using 5% of the labeled data on the DD dataset. From these results, we proved that GraphixMatch can effectively use unlabeled data for training.

4.5. Masking ratios for graph augmentation

We conducted extensive studies on weak and strong augmentations, which are important components of GraphixMatch. We performed experiments to find an appropriate masking ratio by varying masking ratios, which are hyperparameters used for generating augmented graphs. As before, we used node-drop for weak augmentation and subgraph for strong augmentation. We also changed the masking ratios for weak and strong augmentations, respectively, to 0.1, 0.2, 0.3, 0.4, and 0.5. Table 4 shows the experimental results of changing the masking ratios for weak and strong augmentations in GraphixMatch on seven datasets. Table 4 displays the appropriate masking ratios we found to generate the augmented graphs on each dataset. For example, a node-drop masking ratio of 0.1 used for weak augmentation achieved good performance on four datasets: IMDB-B, IMDB-M, REDDIT-B, and REDDIT-M-5K. In each dataset, the masking ratios for strong augmentation performed well at 0.2, 0.3, 0.4, and 0.5. On the PROTEINS dataset, a masking ratio of 0.2 for both node-drop for weak augmentation and subgraph for strong augmentation achieved good performance. On the other hand, the masking ratios of 0.2 and 0.4 of the node-drop for weak augmentation outperformed different masking ratios on the COLLAB and DD datasets, respectively. In both datasets, the masking ratio of 0.1 of subgraph for strong augmentation achieved good performance.

4.6. Thresholds in GraphixMatch

GraphixMatch uses a predefined threshold when generating artificial labels from weakly augmented unlabeled data. The threshold is an important component of GraphixMatch because it can increase confidence in the performance of the model when generating artificial labels. Therefore, we conducted experiments to observe the effect of the threshold for classification performance. We performed experiments by changing the thresholds to 0.25, 0.5, 0.75, 0.85, 0.90, 0.95, 0.97, and 0.99. Table 5 shows the results of applying and changing

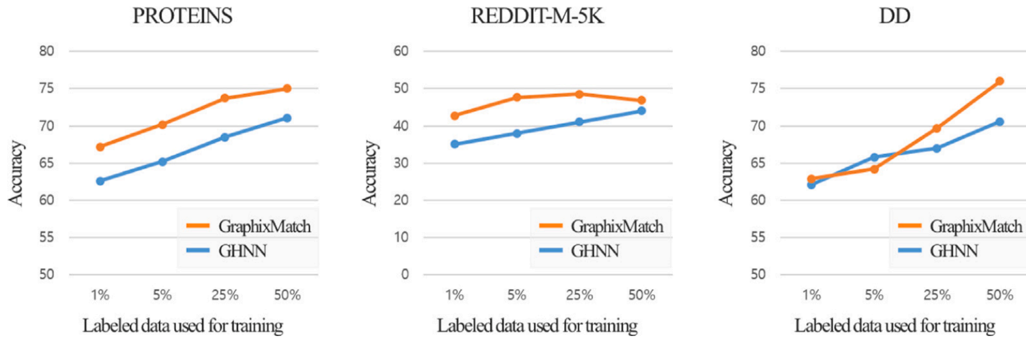

Fig. 3. Plot showing the performance of GraphixMatch and GHNN when varying the proportions of labeled data used for training.

Table 3
Results of GraphixMatch when varying the number of labeled data used for training. The highest accuracy for each dataset is in bold.

Labeled data	PROTEINS	IMDB-B	IMDB-M	REDDIT-B	REDDIT-M-5K	COLLAB	DD
1%	67.2 ± 3.5	63.1 ± 5.5	33.7 ± 1.1	76.6 ± 4.2	42.8 ± 3.9	63.2 ± 3.6	62.9 ± 5.7
5%	70.2 ± 3.9	66.9 ± 5.0	36.3 ± 3.0	79.9 ± 3.3	47.6 ± 1.5	68.9 ± 0.4	64.2 ± 3.8
25%	73.7 ± 2.2	71.4 ± 1.0	42.2 ± 2.9	82.4 ± 2.4	**48.5 ± 1.3**	68.6 ± 0.8	69.7 ± 4.7
50%	**75.0 ± 0.9**	**73.9 ± 1.2**	**45.3 ± 2.0**	**83.8 ± 2.5**	46.8 ± 1.4	**69.2 ± 0.3**	**76.0 ± 5.4**
75%	79.1 ± 4.5	78.4 ± 2.1	50.7 ± 2.9	83.6 ± 2.6	50.0 ± 1.1	70.4 ± 1.4	92.0 ± 1.6
100%	79.5 ± 3.7	79.0 ± 1.7	49.6 ± 2.8	81.7 ± 5.5	50.0 ± 1.4	70.4 ± 2.3	99.7 ± 0.7

Table 4
Results of GraphixMatch when varying the masking ratio, which is a hyperparameter used to generate augmented graphs. The highest accuracy for each dataset is in bold.

Masking ratio		PROTEINS	IMDB-B	IMDB-M	REDDIT-B	REDDIT-M-5K	COLLAB	DD
Weak Aug.	Strong Aug.							
0.1	0.1	72.8 ± 1.3	71.2 ± 2.9	38.9 ± 4.0	81.7 ± 0.6	46.5 ± 2.7	67.5 ± 0.7	72.4 ± 3.4
	0.2	73.1 ± 2.3	**73.9 ± 1.2**	42.7 ± 4.9	80.2 ± 3.8	45.0 ± 2.6	68.9 ± 1.7	71.8 ± 2.1
	0.3	72.8 ± 1.3	73.4 ± 1.9	**45.3 ± 2.0**	81.2 ± 4.4	44.3 ± 2.1	68.4 ± 1.0	73.4 ± 3.5
	0.4	73.3 ± 2.4	72.8 ± 0.8	37.3 ± 3.4	**83.8 ± 2.5**	44.2 ± 4.0	68.5 ± 0.9	73.3 ± 4.3
	0.5	73.3 ± 1.7	72.3 ± 3.2	44.5 ± 3.2	82.9 ± 2.9	**46.8 ± 1.4**	68.9 ± 1.5	72.7 ± 2.9
0.2	0.1	72.2 ± 2.0	72.5 ± 2.4	36.3 ± 2.6	84.2 ± 1.9	46.6 ± 1.7	**69.2 ± 0.3**	73.8 ± 2.8
	0.2	**75.0 ± 0.9**	71.8 ± 2.3	40.1 ± 6.2	82.1 ± 3.6	44.8 ± 4.0	68.5 ± 0.9	73.1 ± 2.7
	0.3	74.8 ± 2.7	70.2 ± 2.5	40.4 ± 5.0	79.3 ± 4.2	45.2 ± 2.1	68.6 ± 1.6	73.7 ± 1.9
	0.4	74.2 ± 1.5	71.7 ± 1.5	39.9 ± 4.8	83.3 ± 1.7	44.8 ± 2.0	68.6 ± 1.0	74.2 ± 2.8
	0.5	72.9 ± 2.2	71.6 ± 1.6	39.7 ± 5.3	82.0 ± 3.5	45.8 ± 1.1	67.8 ± 1.3	73.6 ± 3.8
0.3	0.1	72.6 ± 1.2	66.5 ± 4.0	38.9 ± 6.3	79.5 ± 2.6	42.9 ± 5.0	68.4 ± 0.7	73.6 ± 1.2
	0.2	74.0 ± 3.5	64.4 ± 4.9	37.2 ± 3.5	80.5 ± 2.6	43.8 ± 5.7	67.8 ± 0.6	75.3 ± 3.1
	0.3	73.7 ± 2.7	67.1 ± 5.4	38.7 ± 4.5	82.7 ± 1.9	43.5 ± 2.2	67.1 ± 0.4	74.2 ± 2.7
	0.4	73.7 ± 1.6	67.6 ± 4.7	36.7 ± 4.2	79.4 ± 4.5	44.5 ± 1.7	68.2 ± 1.6	72.0 ± 3.4
	0.5	73.3 ± 1.4	65.0 ± 3.4	36.8 ± 4.2	79.2 ± 2.7	44.9 ± 1.4	68.1 ± 1.7	72.9 ± 1.9
0.4	0.1	72.3 ± 1.4	62.8 ± 4.9	34.9 ± 3.5	80.0 ± 1.8	41.7 ± 6.2	69.1 ± 0.6	**76.0 ± 5.4**
	0.2	69.8 ± 2.8	59.0 ± 3.2	34.0 ± 2.2	79.6 ± 1.9	39.1 ± 3.9	66.8 ± 1.0	75.0 ± 4.5
	0.3	73.5 ± 2.5	59.0 ± 6.0	38.3 ± 3.2	79.2 ± 3.0	44.9 ± 2.1	68.8 ± 1.1	74.4 ± 1.3
	0.4	71.5 ± 2.9	62.0 ± 6.9	33.9 ± 2.3	82.6 ± 1.5	44.1 ± 3.3	67.4 ± 3.3	72.1 ± 3.9
	0.5	72.8 ± 2.4	62.6 ± 4.8	35.3 ± 2.0	79.4 ± 2.8	43.6 ± 2.4	66.9 ± 1.7	73.8 ± 2.3
0.5	0.1	71.7 ± 2.2	52.1 ± 5.6	33.7 ± 1.7	79.0 ± 1.6	42.6 ± 3.6	67.3 ± 1.6	74.4 ± 1.3
	0.2	74.2 ± 2.6	54.9 ± 4.9	32.9 ± 1.7	78.7 ± 1.6	42.0 ± 3.1	67.9 ± 1.9	74.4 ± 5.3
	0.3	72.6 ± 2.9	53.0 ± 4.5	33.3 ± 1.3	80.8 ± 2.6	40.0 ± 4.0	65.9 ± 2.7	74.5 ± 5.5
	0.4	73.1 ± 2.9	54.6 ± 6.5	33.3 ± 2.4	79.2 ± 1.9	38.2 ± 4.2	65.2 ± 2.9	71.4 ± 1.1
	0.5	72.7 ± 4.7	52.4 ± 7.8	34.0 ± 2.5	78.4 ± 3.4	44.5 ± 1.7	67.0 ± 1.5	75.3 ± 4.7

the thresholds required to generate artificial labels in GraphixMatch. In Table 5, GraphixMatch showed little change in performance with changing thresholds. In particular, the four datasets PROTEINS, IMDB-B, IMDB-M, and REDDIT-B showed consistent performance despite the use of different thresholds. Therefore, we observed that the threshold is a hyperparameter that insensitive to GraphixMatch training.

4.7. Ablation study

We conducted an ablation study to examine the impact of weak and strong augmentations on the overall performance of GraphixMatch. Specifically, we performed experiments in which we substituted edge-drop for the node-drop for weak augmentation and the subgraph for strong augmentation. All experiments were conducted by using masking ratios of weak and strong augmentation, which achieved good performance on each dataset in Table 4. Table 6 shows the results of applying different combinations of weak and strong augmentations in GraphixMatch. In Table 6, our default usage in GraphixMatch, which is node-drop for weak augmentation and subgraph for strong augmentation, generally achieved superior performance over other combinations of augmentation strategies on all datasets except REDDIT-M-5K. In particular, except on the REDDIT-M-5K dataset, there was a clear performance difference between using subgraph for strong augmentation and using edge-drop. From these results, we observed that the subgraph defined as strong augmentation in GraphixMatch is effective in generating strongly distorted graphs that preserve the semantic meaning part of the entire graph.

Table 5
Results of GraphixMatch with variations in the thresholds used when generating artificial labels.

Threshold	PROTEINS	IMDB-B	IMDB-M	REDDIT-B	REDDIT-M-5K	COLLAB	DD
0.25	75.0 ± 0.9	73.9 ± 1.2	45.5 ± 2.0	83.8 ± 2.5	45.3 ± 1.0	68.6 ± 1.1	74.5 ± 7.2
0.5	75.0 ± 0.9	73.9 ± 1.2	45.5 ± 2.0	83.8 ± 2.5	44.6 ± 4.5	68.6 ± 1.6	74.5 ± 7.2
0.75	75.0 ± 0.9	73.9 ± 1.2	45.5 ± 2.0	83.8 ± 2.5	45.4 ± 1.7	69.4 ± 1.7	74.5 ± 7.2
0.85	75.0 ± 0.9	73.9 ± 1.2	45.5 ± 2.0	83.8 ± 2.5	46.8 ± 1.9	68.2 ± 1.3	74.4 ± 7.1
0.90	75.0 ± 0.9	73.9 ± 1.2	45.5 ± 2.0	83.8 ± 2.5	46.7 ± 2.1	69.5 ± 1.5	74.4 ± 7.1
0.95 (default)	75.0 ± 0.9	73.9 ± 1.2	45.5 ± 2.0	83.8 ± 2.5	46.8 ± 1.4	69.5 ± 1.5	76.0 ± 5.4
0.97	75.0 ± 0.9	73.9 ± 1.2	45.5 ± 2.0	83.8 ± 2.5	46.1 ± 1.4	69.4 ± 0.3	76.0 ± 5.4
0.99	75.0 ± 0.9	73.9 ± 1.2	45.5 ± 2.0	83.8 ± 2.5	45.3 ± 2.0	69.1 ± 0.5	76.0 ± 5.4

Table 6
Results of GraphixMatch when varying augmentation strategies. The highest accuracy for each dataset is in bold.

Augmentation strategy		PROTEINS	IMDB-B	IMDB-M	REDDIT-B	REDDIT-M-5K	COLLAB	DD
Weak Aug.	Strong Aug.							
Edge-drop	Subgraph	73.6 ± 2.9	71.2 ± 2.4	43.4 ± 5.3	82.0 ± 3.0	46.6 ± 1.5	67.5 ± 1.1	73.2 ± 2.2
Node-drop	Edge-drop	72.2 ± 1.7	68.2 ± 2.6	33.3 ± 1.2	78.1 ± 4.6	39.1 ± 3.7	**69.5 ± 1.7**	72.1 ± 2.9
Node-drop	Subgraph	**75.0 ± 0.9**	**73.9 ± 1.2**	**45.3 ± 2.0**	**83.8 ± 2.5**	**46.8 ± 1.4**	69.2 ± 0.3	**76.0 ± 5.4**

5. Conclusion

In this study, we proposed a GraphixMatch framework to address the scarcity of labeled graph data. GraphixMatch combined graph augmentation strategies with FixMatch which simply achieves good performance in the field of computer vision. We redefined weak and strong augmentations for graphs in GraphixMatch. We demonstrated that GraphixMatch outperformed the supervised baseline and other SSL approaches on seven datasets in experiments with limited labeled graphs. In addition, we demonstrated the effectiveness of weak and strong augmentations using GraphixMatch through extensive experiments. In conclusion, GraphixMatch showed the best performance when applying node-drop for weak augmentation and subgraph for strong augmentation.

Our work can be extended in several interesting directions. First of all, improving the robust SSL approach in class imbalance scenarios makes it easier to apply different datasets in the real world. For example, applying an adaptive threshold for a specific condition provides flexibility for different classes. The adaptive threshold will be able to provide practical training effects by considering the learning status and learning difficulty of each class differently. In addition, open-set SSL approaches can be adopted to overcome scenarios that have class distribution mismatches between labeled and unlabeled data. Regularizing the network for consistency against uncertain noise avoids performance degradation even with unlabeled data containing previously undiscovered classes in the labeled training data. In future work, we hope to develop a method to solve the problems of class imbalance and class distribution mismatches.

CRediT authorship contribution statement

Eunji Koh: Writing – original draft, Methodology, Investigation, Formal analysis, Conceptualization. **Young Jae Lee:** Writing – review & editing, Methodology, Investigation, Conceptualization. **Seoung Bum Kim:** Writing – review & editing, Validation, Supervision, Methodology, Conceptualization.

Declaration of competing interest

We warrant that the manuscript is our original work, has not been published before, and is not currently under consideration for publication elsewhere.

We confirm that there are no known conflicts of interest associated with this publication and there has been no significant financial support for this work that could have influenced its outcome.

Data availability

The data we used is available as an open-source dataset.

Acknowledgments

This research was supported by Brain Korea 21 FOUR, the Ministry of Science and ICT (MSIT) in Korea under the ITRC support program supervised by the Institute for Information Communication Technology Planning and Evaluation (IITP-2020-0-01749), and the National Research Foundation of Korea grant funded by the Korea government (RS-2022-00144190).

References

[1] V. La Gatta, V. Moscato, M. Postiglione, G. Sperli, An epidemiological neural network exploiting dynamic graph structured data applied to the covid-19 outbreak, IEEE Trans. Big Data 7 (1) (2020) 45–55.
[2] D. Zhang, J. Yin, X. Zhu, C. Zhang, Network representation learning: A survey, IEEE Trans. Big Data 6 (1) (2018) 3–28.
[3] H.-J. Moon, S.-J. Bu, S.-B. Cho, A graph convolution network with subgraph embedding for mutagenic prediction in aromatic hydrocarbons, Neurocomputing 530 (2023) 60–68.
[4] Y. Xie, Y. Liang, M. Gong, A. Qin, Y.-S. Ong, T. He, Semisupervised graph neural networks for graph classification, IEEE Trans. Cybern. (2022).
[5] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, S.Y. Philip, A comprehensive survey on graph neural networks, IEEE Trans. Neural Netw. Learn. Syst. 32 (1) (2020) 4–24.
[6] T. Ma, W. Shao, Y. Hao, J. Cao, Graph classification based on graph set reconstruction and graph kernel feature reduction, Neurocomputing 296 (2018) 33–45.
[7] H. Xu, S. Zhang, B. Jiang, J. Tang, Graph context-attention network via low and high order aggregation, Neurocomputing 536 (2023) 152–163.
[8] C. Lu, Q. Liu, C. Wang, Z. Huang, P. Lin, L. He, Molecular property prediction: A multilevel quantum interactions modeling perspective, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 33, 2019, pp. 1052–1060.
[9] Y. Xie, S. Lv, Y. Qian, C. Wen, J. Liang, Active and semi-supervised graph neural networks for graph classification, IEEE Trans. Big Data (2022).
[10] J. Ma, R.P. Sheridan, A. Liaw, G.E. Dahl, V. Svetnik, Deep neural nets as a method for quantitative structure–activity relationships, J. Chem. Inf. Model. 55 (2) (2015) 263–274.
[11] J. Gilmer, S.S. Schoenholz, P.F. Riley, O. Vinyals, G.E. Dahl, Neural message passing for quantum chemistry, in: International Conference on Machine Learning, PMLR, 2017, pp. 1263–1272.
[12] J. Lee, I. Lee, J. Kang, Self-attention graph pooling, in: International Conference on Machine Learning, PMLR, 2019, pp. 3734–3743.
[13] Z. Ying, J. You, C. Morris, X. Ren, W. Hamilton, J. Leskovec, Hierarchical graph representation learning with differentiable pooling, Adv. Neural Inf. Process. Syst. 31 (2018).
[14] X. Wang, D. Bo, C. Shi, S. Fan, Y. Ye, P.S. Yu, A survey on heterogeneous graph embedding: Methods, techniques, applications and sources, IEEE Trans. Big Data (2022) 1.

[15] Y. Zhang, X. Wang, C. Shi, X. Jiang, Y. Ye, Hyperbolic graph attention network, IEEE Trans. Big Data 8 (6) (2022) 1690–1701.
[16] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.
[17] M. Welling, T.N. Kipf, Semi-supervised classification with graph convolutional networks, in: J. International Conference on Learning Representations, ICLR 2017, 2016.
[18] K. Xu, W. Hu, J. Leskovec, S. Jegelka, How powerful are graph neural networks? in: International Conference on Learning Representations, 2019.
[19] R.G. Parr, Density functional theory, Annu. Rev. Phys. Chem. 34 (1) (1983) 631–656.
[20] D. Berthelot, N. Carlini, E.D. Cubuk, A. Kurakin, K. Sohn, H. Zhang, C. Raffel, ReMixMatch: Semi-supervised learning with distribution matching and augmentation anchoring, in: International Conference on Learning Representations, 2019.
[21] D. Berthelot, N. Carlini, I. Goodfellow, N. Papernot, A. Oliver, C.A. Raffel, Mixmatch: A holistic approach to semi-supervised learning, Adv. Neural Inf. Process. Syst. 32 (2019).
[22] Y. Grandvalet, Y. Bengio, Semi-supervised learning by entropy minimization, Adv. Neural Inf. Process. Syst. 17 (2004).
[23] T. Miyato, S.-i. Maeda, M. Koyama, S. Ishii, Virtual adversarial training: a regularization method for supervised and semi-supervised learning, IEEE Trans. Pattern Anal. Mach. Intell. 41 (8) (2018) 1979–1993.
[24] K. Sohn, D. Berthelot, N. Carlini, Z. Zhang, H. Zhang, C.A. Raffel, E.D. Cubuk, A. Kurakin, C.-L. Li, Fixmatch: Simplifying semi-supervised learning with consistency and confidence, Adv. Neural Inf. Process. Syst. 33 (2020) 596–608.
[25] A. Tarvainen, H. Valpola, Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results, Adv. Neural Inf. Process. Syst. 30 (2017).
[26] D.-H. Lee, et al., Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks, in: Workshop on Challenges in Representation Learning, ICML, Vol. 3, 2013, p. 896.
[27] P. Bachman, O. Alsharif, D. Precup, Learning with pseudo-ensembles, Adv. Neural Inf. Process. Syst. 27 (2014).
[28] W. Ju, X. Luo, Z. Ma, J. Yang, M. Deng, M. Zhang, GHNN: Graph harmonic neural networks for semi-supervised graph-level classification, Neural Netw. 151 (2022) 70–79.
[29] F.-Y. Sun, J. Hoffman, V. Verma, J. Tang, InfoGraph: Unsupervised and semi-supervised graph-level representation learning via mutual information maximization, in: International Conference on Learning Representations, 2020.
[30] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, Y. Shen, Graph contrastive learning with augmentations, Adv. Neural Inf. Process. Syst. 33 (2020) 5812–5823.
[31] K.M. Borgwardt, C.S. Ong, S. Schönauer, S. Vishwanathan, A.J. Smola, H.-P. Kriegel, Protein function prediction via graph kernels, Bioinformatics 21 (suppl_1) (2005) i47–i56.
[32] P. Yanardag, S. Vishwanathan, Deep graph kernels, in: Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2015, pp. 1365–1374.
[33] P.D. Dobson, A.J. Doig, Distinguishing enzyme structures from non-enzymes without alignments, J. Mol. Biol. 330 (4) (2003) 771–783.
[34] Y. You, T. Chen, Y. Shen, Z. Wang, Graph contrastive learning automated, in: International Conference on Machine Learning, PMLR, 2021, pp. 12121–12132.
[35] N.E. Khalifa, M. Loey, S. Mirjalili, A comprehensive survey of recent trends in deep learning for digital images augmentation, Artif. Intell. Rev. (2021) 1–27.
[36] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, 2014, arXiv preprint arXiv:1412.6980.
[37] M. Fey, J.E. Lenssen, Fast graph representation learning with PyTorch geometric, 2019, arXiv preprint arXiv:1903.02428.

Eunji Koh received a B.S. in 2021 and is currently a M.S. student in the School of Industrial and Management Engineering, Korea University, Seoul, Korea. Her research interests include semi-supervised learning, multichannel signal analysis and their industrial applications.

Young Jae Lee received a B.S. in 2019 and is currently a Ph.D. candidate in the School of Industrial and Management Engineering, Korea University, Seoul, Korea. His research interests include deep reinforcement learning, semi-supervised learning and self-supervised learning.

Seoung Bum Kim is a Professor in the School of Industrial and Management Engineering at Korea University. He serves as Director of Artificial Intelligence Engineering Center in Korea University. From 2005–2009, he was an Assistant Professor in the Department of Industrial & Manufacturing Systems Engineering at the University of Texas at Arlington. He received an M.S. in Industrial and Systems Engineering in 2001 and a Ph.D. in Industrial and Systems Engineering in 2005 from the Georgia Institute of Technology. His research interests use machine learning algorithms to create new methods for various problems appearing in engineering and science. He has published more than 180 internationally recognized journals and refereed conference proceedings. He is actively involved with the INFORMS, serving as president for the INFORMS Section on Data Mining.